

CampusPark

Smart Campus Parking Reservation System

Final Project Report · Phase II

Guanya Song · April 2026

1. Project Overview

CampusPark is a web application for reserving campus parking spots around the Seattle area. Students and visitors can search available lots, book a spot in advance, check in at the gate with a 6-digit ticket code, and check out with automatic fee calculation. Administrators get a dashboard showing occupancy, revenue, and user activity over time.

The motivation came from how hard it is to find parking on campus when you arrive without knowing which lots are full. Most systems only show a static map. This project tries to fix that by showing live availability and helping users pick a lot based on where they are coming from.

The three main parts of the system are:

- **Live availability** — spot counters update every time a reservation is created, checked in, or cancelled. A background job also simulates realistic occupancy changes every 15 minutes.
- **Route-based recommendation** — the user enters a departure address or ZIP code, the server geocodes it, and scores each lot by distance, price, and availability to suggest the best option.
- **Reservation lifecycle** — a booking starts as PENDING, moves to ACTIVE after ticket check-in, and closes as COMPLETED at checkout. Unclaimed PENDING reservations expire after 5 minutes so the spot becomes available again.

2. System Architecture

2.1 UML Component Diagram

Figure 1 shows the UML Component Diagram produced for Assignment 5. The system is structured across four tiers: Browser SPA, Node.js Backend, Prisma ORM, and PostgreSQL Database.

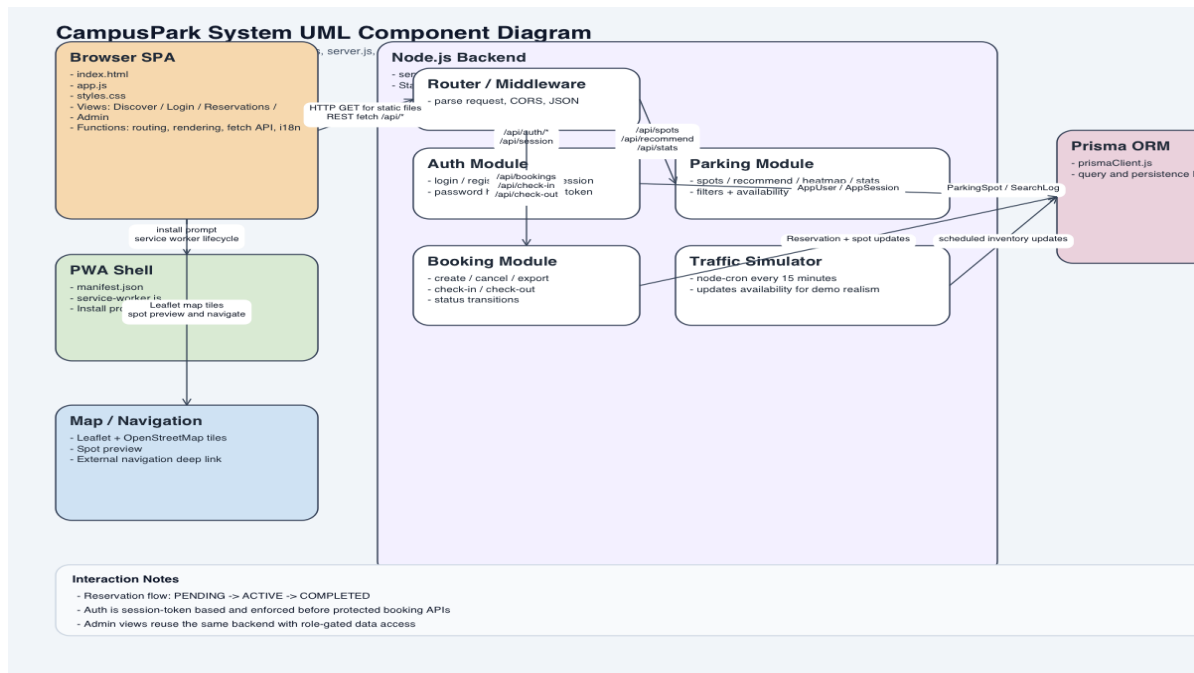


Figure 1. CampusPark UML Component Diagram (Assignment 5).

2.2 Component Descriptions

Component	Files	Responsibility
Browser SPA	index.html, app.js, styles.css	Client-side routing, all view rendering, funnel analytics batching, feature-flag loader, data
PWA Shell	manifest.json, service-worker.js	Home-screen installation on iOS/Android; caches application shell.
Map / Navigation	Leaflet + OSRM + Nominatim	OpenStreetMap tiles, MarkerCluster aggregation, Leaflet.heat demand heatmap, driving
Node.js Backend	server.js (~2,000 lines)	REST API routing, static file serving, script auth, rate limiting, structured JSON logging
Prisma ORM	prisma/schema.prisma	Type-safe DB access. Six models: AppUser, AppSession, ParkingSpot, Reservation,
PostgreSQL	Hosted via DATABASE_URL	Persistent state. Atomic LEAST() updates prevent counter overflow.

3. Core Features

3.1 Search and Reservation

Users can filter by keyword, zone, arrival time, duration, and EV-only. The Recommend button scores all spots on the server and highlights the best match on the map. The booking form validates plate and phone format before submitting. Idempotency keys prevent a double booking if the form is submitted twice due to a slow network.

3.2 Route-Based Parking Assistant

The departure input accepts a full address or a 5-digit ZIP code. If the user enters a ZIP, the frontend appends the state and country before geocoding (e.g. 98006 becomes 98006, WA, USA). Address autocomplete shows Nominatim suggestions with a 320 ms debounce. Five ZIP chips are shown below the input for common Seattle-area locations.

3.3 Analytics Dashboard

The admin panel shows hourly booking counts and revenue by zone, a 6-step user funnel (search through check-out) with conversion rates, DAU/WAU/MAU counts, and a button to download all reservations as a CSV.

3.4 Feature Flags

Six feature flags are stored in the database and cached in memory for 30 seconds. Admins can toggle them without restarting the server.

3.5 Other UX Details

Dark mode with localStorage persistence, an onboarding dialog shown once on first visit, QR code on checked-in booking cards, live countdown timers on pending and active reservations, and star ratings on completed bookings.

4. Key Algorithms

Algorithm	Description
Dynamic Pricing	Price = pricePerHour × Dh × Bs. Dh = 24-point hourly demand curve, peaks at 8 AM and 5 PM (weekdays). Bs = deterministic per-spot hash offset in [0.75, 1.25]. EV spaces carry a higher base pricePerHour set at seed time.
Surge Pricing	If availableSpots / totalSpots < 0.30, apply 1.5× multiplier. Original price shown crossed out in the UI.
Recommendation Score	score = 0.5·sprox + 0.3·savail + 0.2·scrowd sprox: degrades linearly from 1 (on-route ≤300 m) to 0 (2 km). savail: min(available/10, 1). scrowd: 1 – timeDemand × popularityFactor (lower crowd = better). Only spots with available > 0 included. Top-3 returned.
Expiry Cron	Every 60 s: find PENDING reservations older than 5 min, set EXPIRED, restore counter via LEAST(available+1, total).
Funnel Analytics	6 events batched every 10 s (keepalive fetch on visibilitychange). Stored in FunnelEvent; 7-day conversion rates computed server-side.

5. Limitations

- 1. Simulated occupancy.** Availability numbers come from a cron-based simulator, not from real gate sensors. The trends look realistic but the counts do not match what is actually happening in any real lot.
- 2. Seattle only.** ZIP normalisation hard-codes the state as Washington. Deploying for a campus in another state would require changing that.
- 3. No real payments.** The checkout flow calculates a fee and stores it, but there is no integration with a payment processor.
- 4. In-process rate limiting.** Rate limit counters live in Node.js memory. Multiple server instances behind a load balancer would need Redis.
- 5. Simple session model.** Sessions are random tokens in the database with no expiry or revocation beyond deleting the row manually.

6. No accessibility audit. The app has not been tested with a screen reader or run through an automated WCAG check.

6. Future Work

1. Replace the simulator with actual gate/sensor data using a WebSocket connection so the availability counts are real.
2. Add a payment step using Stripe so users are charged at checkout, with refunds handled automatically on cancellation.
3. Build push notifications so users get reminded before their parking window starts or when their booking is about to expire.
4. Support multiple campuses by making the geographic region configurable rather than hard-coded to Seattle.
5. Add a permit tier so students with annual permits can book from a separate reserved pool before the general inventory opens.
6. Do a proper accessibility pass — fix focus order, test with VoiceOver and NVDA, check colour contrast ratios.
7. Use the FunnelEvent and SearchLog data to train a simple demand forecasting model that could warn admins before lots fill up.

7. Conclusion

CampusPark ended up covering more ground than I initially planned. The core reservation workflow — search, book, check in, check out — works end to end, and the admin dashboard gives a decent view of what is happening in the system. The main things I would improve with more time are replacing the simulated data with real sources and adding proper payment handling.

Source code: <https://github.com/sguanya-stack/campus-park>

References

- [1] OpenStreetMap contributors. Nominatim. <https://nominatim.org/>, accessed April 2026.
- [2] Project OSRM contributors. Open Source Routing Machine. <https://project-osrm.org/>, accessed April 2026.
- [3] Agafonkin, V. et al. Leaflet. <https://leafletjs.com/>, accessed April 2026.
- [4] Prisma Data, Inc. Prisma ORM. <https://www.prisma.io/docs/>, accessed April 2026.
- [5] Kelektiv. node-cron. <https://github.com/kelektiv/node-cron>, accessed April 2026.
- [6] Chart.js contributors. Chart.js. <https://www.chartjs.org/>, accessed April 2026.